ectrans on AMD GPUs: OpenMP Target generated from OpenACC

One OpenACC source, OpenMP Target produced at build time

Philippe Marguinaud (Météo-France)

2026

ECMWF / ACCORD

Starting point and goal

- **ectrans 2.0.0** GPU code: OpenACC for NVIDIA, and a hand-written OpenMP Target copy for AMD
  - `#ifdef ACCGPU` / `#ifdef OMPGPU` pairs around every directive
  - two codes to keep in sync; differences hard to see
- **Goal:** a single OpenACC source; the OpenMP Target is **generated** at build time
  - NVIDIA build unchanged
  - results bit-identical, no performance loss on AMD
- **Scope:** the path of the benchmark (`inv_trans`, `dir_trans`)
  - outside: adjoints, `gpnorm_trans`, setup, allocators, `tpm_hicfft`
- **Machines:** A100 (ECMWF, nvfortran 26.3); MI210, MI250, MI300A (amdflang, AFAR 24.3)

1. Method: a converter in the build



NVIDIA: the generated file is compiled by nvfortran as before (no conversion)

- `acc2omp.pl`: Perl, no dependency; run by CMake when `GPU_OFFLOAD=OMP`
- Translates **directives only**, no Fortran parsing
- **Same number of lines** in and out: kernel names keep their line numbers
  - `__omp_offloading_..._trltomP..._l169`: profiles comparable kernel by kernel
- Any directive outside the subset is an **error**: nothing is translated silently
- Hand-written pairs are still possible where the code differs (CUDA/HIP streams)

1. Method: small, verified steps

- **Tools** (`tools/acc2omp`, with tests):
  - `acc2omp.pl --check`: subset checker; `--compare-pairs`: translated OpenACC vs hand-written OpenMP
  - `add_present.pl`: explicit `PRESENT` clauses; `unpair.pl`: removes one pair, keeping the lines
- **Steps:** OpenACC made conform (checked on A100) → converter → CMake → pair removal
- **37 batches** of about 5 directives, one commit each, checked automatically:
  - `cpp -DACCGPU` output unchanged (OpenACC code untouched), `--check` clean, build
- **Every 5 batches:** MI210, MI250, MI300A, 1 and 2 MPI ranks
  - checksums **bit-identical** to the reference (`-c 100`)
  - rocprofv3: same kernels, same call counts, time per kernel
- AMD prerequisites: one GPU per MPI task, `hipMalloc` buffers for GPU-aware MPI, a compiler workaround

2. Constraints on the OpenACC: the subset

Directive	Allowed clauses
DATA	PRESENT COPYIN COPYOUT COPY CREATE IF ASYNC
ENTER / EXIT DATA	COPYIN CREATE / DELETE COPYOUT, IF ASYNC
UPDATE	HOST DEVICE IF ASYNC
HOST_DATA	USE_DEVICE
PARALLEL LOOP	COLLAPSE or TILE, PRIVATE FIRSTPRIVATE DEFAULT(NONE) ASYNC, data clauses
LOOP SEQ, WAIT	

- Rejected: `KERNELS`, `SERIAL`, `ROUTINE`, `DECLARE`, `ATOMIC`, `CACHE`, `LOOP GANG/VECTOR`, ...

2. Constraints on the OpenACC: how to write it

- `PARALLEL LOOP`, not `KERNELS`: the loop is declared independent
  - `KERNELS` made 2 kernels in `trltom`/`trmtol` (copy through a temporary)
- **Explicit data** on every `PARALLEL LOOP`: `PRESENT(...)` for all arrays
  - no reliance on implicit mapping; added with `add_present.pl`
- Scalars: `FIRSTPRIVATE`, not `COPYIN` (kernel argument, no transfer)
- Loop indices never in `FIRSTPRIVATE` (OpenMP forbids it: amdflang rejects it, nvfortran accepts it)
- Optional arguments and arrays mapped under a condition: `DATA ... IF(PRESENT(X))`
  - the converter removes them from `MAP(PRESENT)` on the loops inside
- MPI on device buffers: `HOST_DATA USE_DEVICE(...)`

3. The generated OpenMP: principles

OpenACC	OpenMP Target
DATA / HOST_DATA USE_DEVICE	TARGET DATA / TARGET DATA USE_DEVICE_ADDR
ENTER / EXIT DATA, UPDATE	TARGET ENTER / EXIT DATA, TARGET UPDATE
PARALLEL LOOP	TARGET TEAMS DISTRIBUTE PARALLEL DO
PRESENT / COPYIN / COPYOUT	MAP(PRESENT,ALLOC: / TO: / FROM:
COPY / CREATE / DELETE	MAP(TOFROM: / ALLOC: / DELETE:
UPDATE HOST / DEVICE	FROM / TO
TILE(a,b)	COLLAPSE(2)
ASYNC, WAIT, DEFAULT(NONE), LOOP	dropped

- **Synchronous** OpenMP; `IF`, `PRIVATE`, `FIRSTPRIVATE`, `COLLAPSE` copied as they are

3. The generated OpenMP: an example (trltom)

Before: hand-written pair

```
#ifdef OMPGPU
!$OMP TARGET TEAMS DISTRIBUTE PARALLEL DO &
!$OMP& DEFAULT(NONE) SHARED(PFBUF,PFBUF_IN,...)
DO JPOS=FROM_SEND,TO_SEND
  PFBUF(JPOS-FROM_SEND+FROM_RECV) = PFBUF_IN(JPOS)
ENDDO
!$OMP END TARGET TEAMS DISTRIBUTE PARALLEL DO
#endif
#ifdef ACCGPU
!$ACC PARALLEL LOOP DEFAULT(NONE) &
!$ACC& PRESENT(PFBUF,PFBUF_IN) &
!$ACC& FIRSTPRIVATE(FROM_RECV,...) ASYNC(1)
DO JPOS=FROM_SEND,TO_SEND
  PFBUF(JPOS-FROM_SEND+FROM_RECV) = PFBUF_IN(JPOS)
ENDDO
#endif
```

Now: source (OpenACC only)

```
!$ACC PARALLEL LOOP DEFAULT(NONE) &
!$ACC& PRESENT(PFBUF,PFBUF_IN) &
!$ACC& FIRSTPRIVATE(FROM_RECV,...) ASYNC(1)
DO JPOS=FROM_SEND,TO_SEND
  PFBUF(JPOS-FROM_SEND+FROM_RECV) = PFBUF_IN(JPOS)
ENDDO
```

Generated by acc2omp.pl (same lines):

```
!$OMP TARGET TEAMS DISTRIBUTE PARALLEL DO &
!$OMP& MAP(PRESENT,ALLOC:PFBUF,PFBUF_IN) &
!$OMP& FIRSTPRIVATE(FROM_RECV,...)
DO JPOS=FROM_SEND,TO_SEND
  PFBUF(JPOS-FROM_SEND+FROM_RECV) = PFBUF_IN(JPOS)
ENDDO
```

4. Performance: OpenACC on A100 (nvfortran)

Change of the OpenACC source	Effect (kernel times, nsys)
KERNELS → PARALLEL LOOP (trltom, trmtol)	2 kernels → 1; local copy 2x faster; kernel time −5 % (2 ranks)
Scalar COPYIN → FIRSTPRIVATE	no transfer before each launch; no visible change
Explicit PRESENT clauses	all kernels within 0.95–1.03: noise
PRESENT for conditional arrays	8 configurations bit-identical; no change

- Results **bit-identical** after every step
- Pair removal only moves OpenACC lines (and `KMLOC`: `FIRSTPRIVATE` → `PRIVATE` in `vdtuv`): not run again on A100

4. Performance: generated OpenMP on AMD

s / iteration	hand-written OpenMP	generated OpenMP	change
MI210	0.157 0.121	0.150 0.112	**-4 %** **-7 %**
MI250	0.221 0.141	0.212 0.135	**-4 %** **-4 %**
MI300A	0.090 0.071	0.088 0.065	**-2 %** **-8 %**

upper bar: 1 MPI rank, lower bar: 2 MPI ranks (1 GPU per rank); lower is better

- Checksums **bit-identical**; same kernels, same call counts
- Single kernels within ±5–12 % (e.g. `fsc` −6 to −12 %, `ledir` l387 +6 to +12 %)
- Host → device copies per run: 41678 → 38586 (scalars passed as kernel arguments)
- Kernels are only 20–25 % of the iteration on MI210/MI250: the transfers between `inv_trans` and `dir_trans` dominate

Status and next steps

- **212** OpenACC directives translated at build time; their OpenMP copies removed
- **169** remain in hand-written pairs:
  - outside the scope: adjoints, `gpnorm_trans`, setup, allocators, `tpm_hicfft`
  - 11 pairs with different code on each side (streams in `ledir`, `leinv`, `trgtol`, `trltog`)
- Branch `openacc-subset` (on ectrans 2.0.0, 54 commits, 51 files):
  - `github.com/pmarguinaud/ectrans/tree/openacc-subset`
- Done with an AI coding assistant (Claude Code): tools, conversion, tests, profiling; every step reviewed
- **Possible next steps:**
  - conformance suite for the converter (OpenACC subset vs OpenMP output)
  - extend the scope: adjoints, `gpnorm_trans`
  - asynchronous execution in the generated OpenMP
  - contribute upstream (ECMWF ectrans)